assign6

May 18, 2025

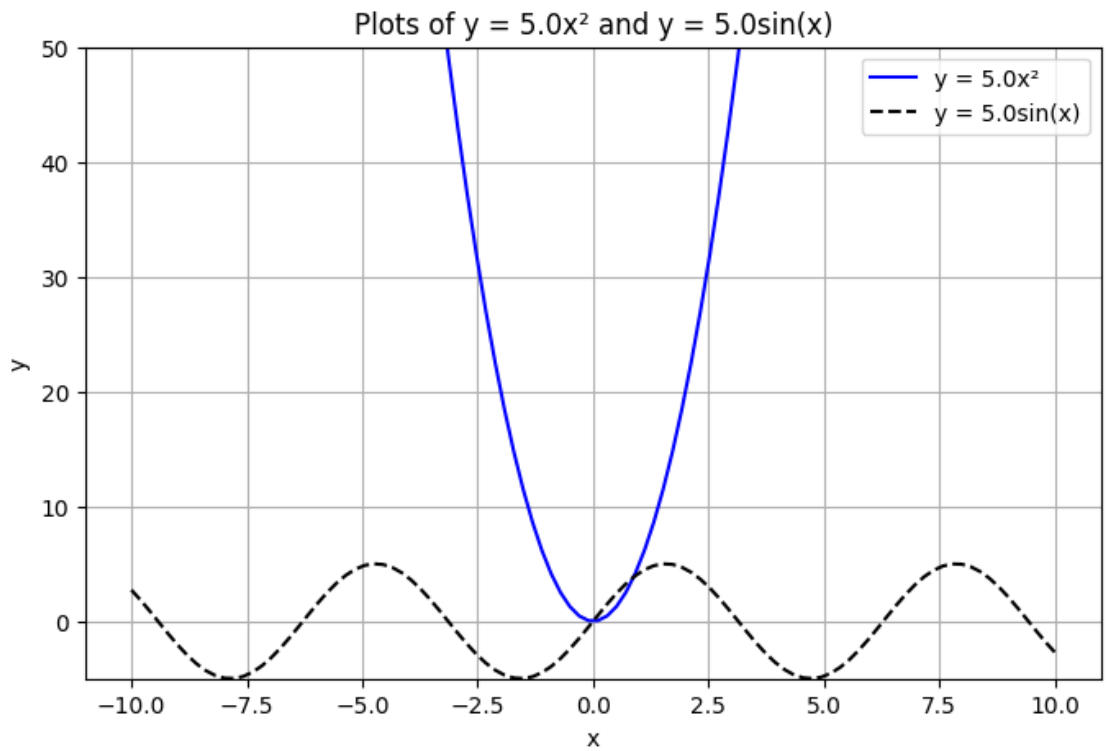
[ ]:

```

```

Q1. Ask the user to enter a value (e.g., M) for a mathematical function. Generate x values from -10 to 10 using np.linspace(). Compute y values for: o y= M x2 o y=M sin(x) Plot both functions in a single figure: o Use different colors and line styles. o Add legend, grid, and title.

[11]:

```
import numpy as np
import matplotlib.pyplot as plt

M = float(input("Enter a value for M: "))

x = np.linspace(-10, 10, 100)

y1 = M * x**2
y2 = M * np.sin(x)

plt.figure(figsize=(8, 5))
plt.plot(x, y1, color='blue', linestyle='-', label=f'y = {M}x²')
plt.plot(x, y2, color='black', linestyle='--', label=f'y = {M}sin(x)')

plt.ylim(-5, 50);

plt.legend()
plt.grid(True)
plt.title(f'Plots of y = {M}x² and y = {M}sin(x)')
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```

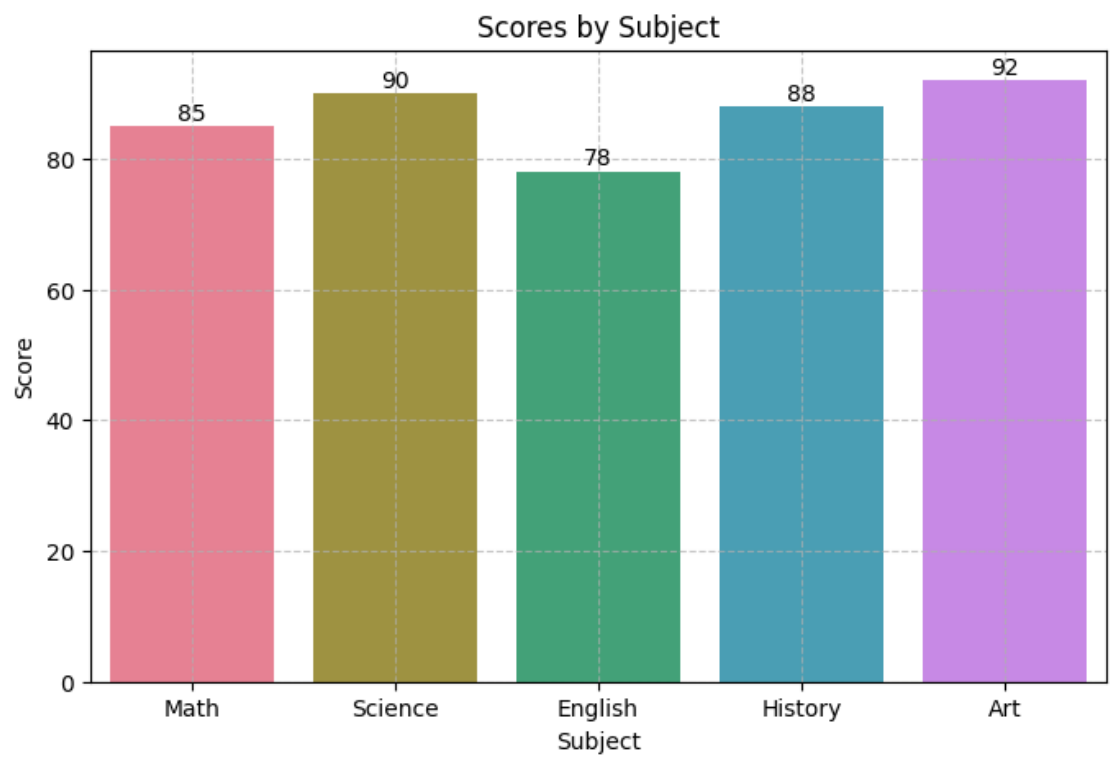


Q.2 Create a dataset of five subjects and scores. Convert it into a Pandas DataFrame. Plot the scores using a Seaborn bar plot with: o Different colors for each bar. o Annotations on top of each bar. o Title, axis labels, and grid.

[12]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {
    'Subject': ['Math', 'Science', 'English', 'History', 'Art'],
    'Score': [85, 90, 78, 88, 92]
}

df = pd.DataFrame(data)

plt.figure(figsize=(8, 5))
colors = sns.color_palette('husl', len(df))
plot = sns.barplot(x='Subject', y='Score', data=df, palette=colors)

for index, row in enumerate(df.iterrows()):
    plt.text(index, row[1].Score + 1, str(row[1].Score), ha='center')

plt.title('Scores by Subject')
```

```
plt.xlabel('Subject')
plt.ylabel('Score')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

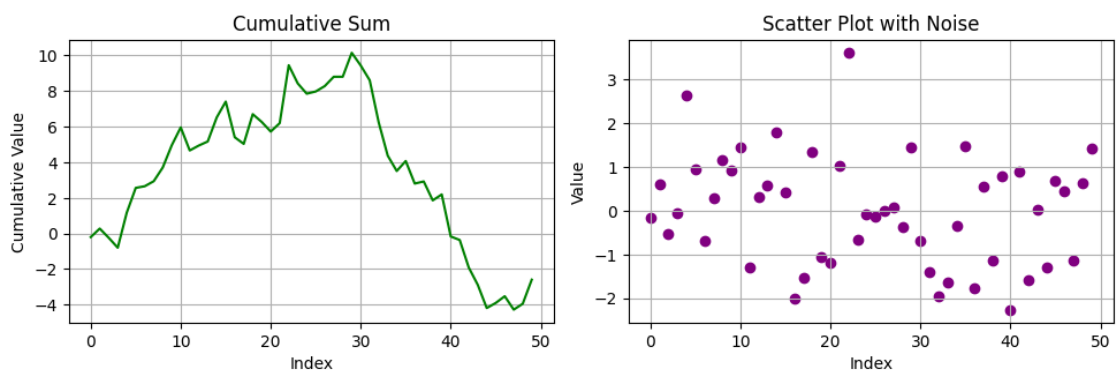
```
C:\Users\Jahid\AppData\Local\Temp\ipykernel_16756\391546344.py:14:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same
effect.

  plot = sns.barplot(x='Subject', y='Score', data=df, palette=colors)
```



Q3. Write a Python script to select your roll number as seed for NumPy and then generate a dataset of 50 values using np.random.randn(). Create a 2x2 subplot layout displaying: o Line plot showing cumulative sum. o Scatter plot with random noise. Customize with titles, axis labels, and grids.

[ ]:
```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(12345)
```

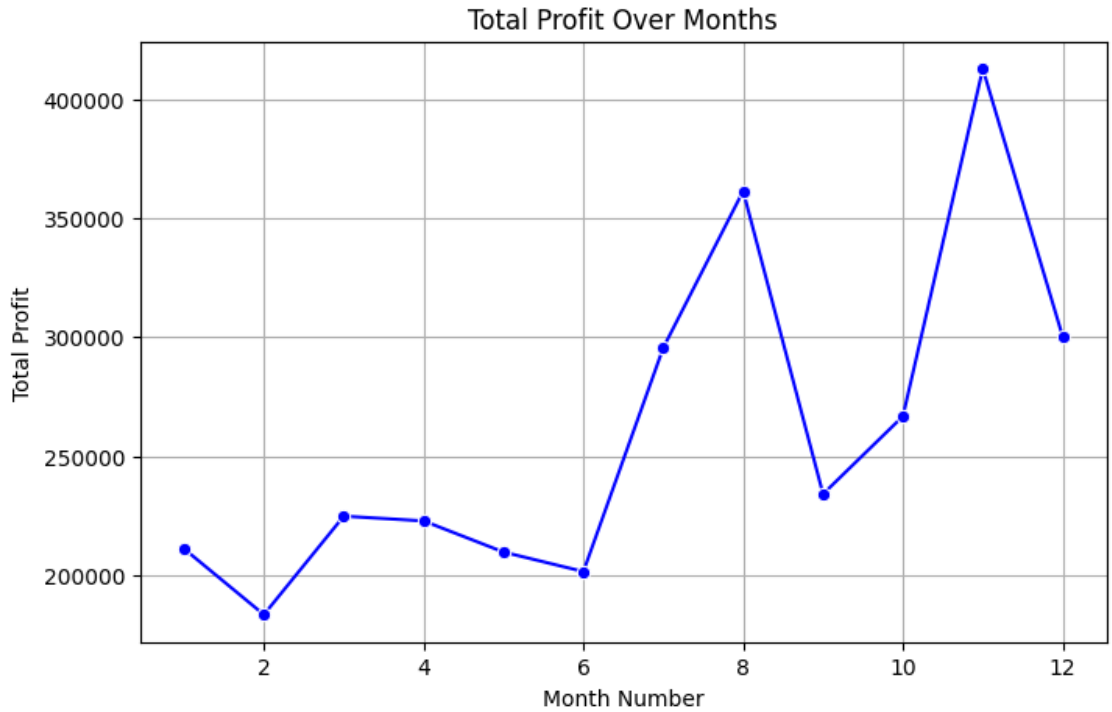
```
data = np.random.randn(50)

plt.figure(figsize=(10, 6))

plt.subplot(2, 2, 1)
plt.plot(np.cumsum(data), color='green')
plt.title('Cumulative Sum')
plt.xlabel('Index')
plt.ylabel('Cumulative Value')
plt.grid(True)

plt.subplot(2, 2, 2)
x = np.arange(50)
y = data + np.random.normal(scale=0.5, size=50)
plt.scatter(x, y, color='purple')
plt.title('Scatter Plot with Noise')
plt.xlabel('Index')
plt.ylabel('Value')
plt.grid(True)

plt.tight_layout()
plt.show()
```

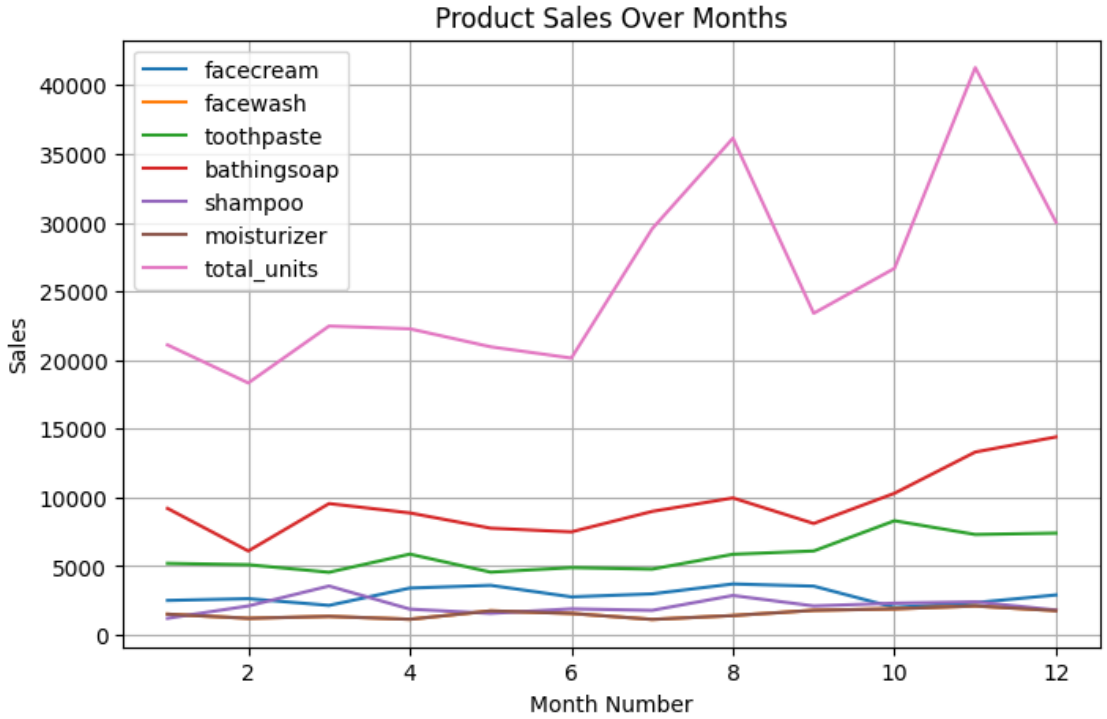


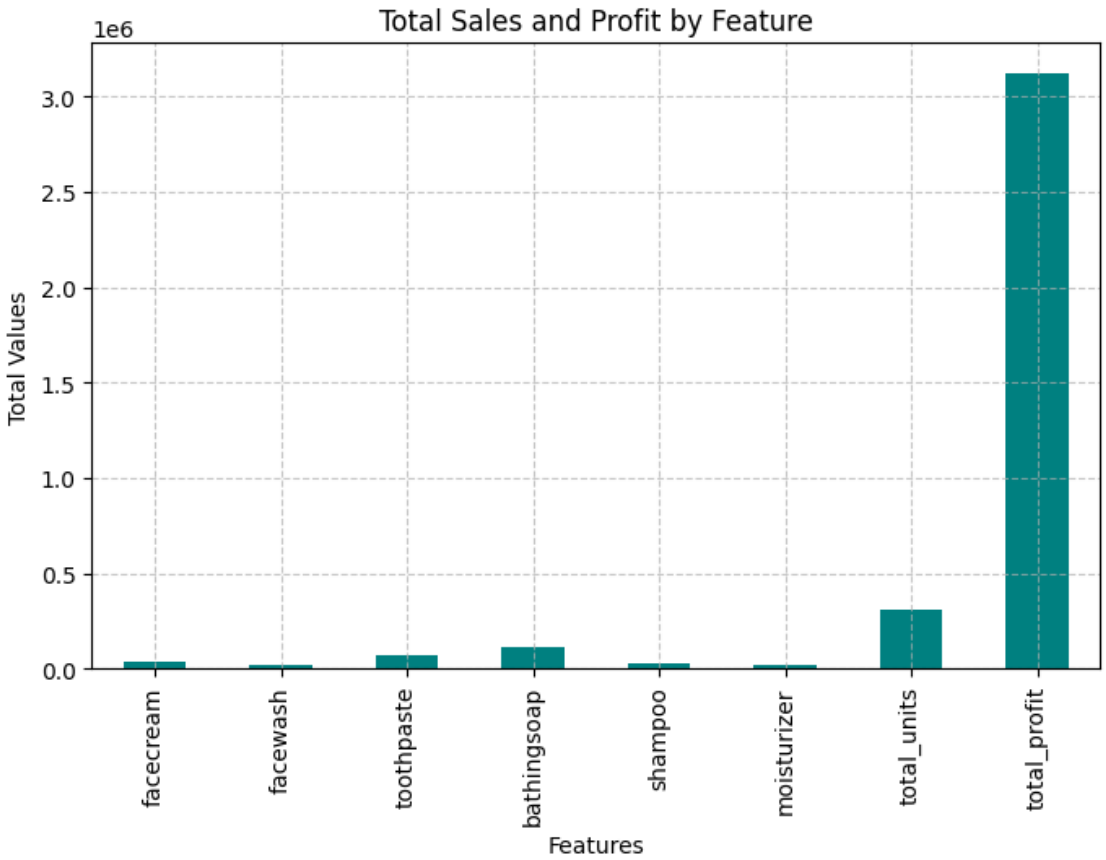
Q.4 Download Data-set from the below link https://github.com/AnjulaMehto/MCA/blob/main/company_sales_d
Apply 'seaborn' library to do the following. 1. Read Total profit of all months and show it using a line plot. 2. Read all product sales data and show it using a multiline plot. 3. Plot bar chart for all the features/attributes.

[ ]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("company_sales_data.csv")
plt.figure(figsize=(8, 5))
sns.lineplot(x=df['month_number'], y=df['total_profit'], marker='o',␣
 ↪color='blue')
plt.title('Total Profit Over Months')
plt.xlabel('Month Number')
plt.ylabel('Total Profit')
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 5))
for column in df.columns[1:-1]:
    sns.lineplot(x=df['month_number'], y=df[column], label=column)
plt.title('Product Sales Over Months')
plt.xlabel('Month Number')
plt.ylabel('Sales')
plt.grid(True)
plt.legend()
plt.show()

plt.figure(figsize=(8, 5))
df.sum().drop('month_number').plot(kind='bar', color='teal')
plt.title('Total Sales and Profit by Feature')
plt.xlabel('Features')
plt.ylabel('Total Values')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```


Total Profit Over Months
Total Profit
400000
350000
300000
250000
200000
2
4
6
8
10
12
Month Number


Product Sales Over Months
facecream
facewash
toothpaste
bathingsoap
shampoo
moisturizer
total_units
Sales
40000
35000
30000
25000
20000
15000
10000
5000
0
2
4
6
8
10
12
Month Number


Total Sales and Profit by Feature
1e6
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Total Values
facecream
facewash
toothpaste
bathingsoap
shampoo
moisturizer
total_units
total_profit
Features

Assignment7

May 18, 2025

[8]:
```
import numpy as np
import pandas as pd

rn=102317036
np.random.seed(rn)
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct",␣
↪"Nov", "Dec"]
categories = ["Electronics", "Clothing", "Home & Kitchen", "Sports"]

sales_data=np.random.randint(1000, 5001, size=(12, 4))

df = pd.DataFrame(sales_data, columns=categories, index=months)

print(df.head(5))
print(df.describe())


totals = df.sum()
print(totals)
totalm = df.sum(axis=1)
print(totalm)
growth_rates = df.pct_change().mean() * 100


df["Total Sales"] = totalm
df["Growth Rate"] = df["Total Sales"].pct_change() * 100

print(df)

discounted_df = df.copy()
if rn % 2 == 0:
    discounted_df["Electronics"] *= 0.9
else:
    discounted_df["Clothing"] *= 0.85

discounted_df
```

```
     Electronics  Clothing  Home & Kitchen  Sports
```

```
Jan         3408      3311            1987    3174
Feb         3152      3458            3713    2427
Mar         4286      4058            2713    1383
Apr         3325      4882            4910    1150
May         2369      1158            2155    1905
       Electronics     Clothing  Home & Kitchen       Sports
count    12.000000    12.000000        12.00000    12.000000
mean   2920.916667  3445.916667      3237.25000  2747.916667
std     905.324599  1212.336171      1121.92579  1197.909578
min    1499.000000  1158.000000      1090.00000  1150.000000
25%    2323.750000  2625.500000      2573.50000  1774.500000
50%    2907.000000  3492.500000      3359.50000  2588.000000
75%    3518.500000  4546.250000      3870.50000  3694.750000
max    4286.000000  4882.000000      4910.00000  4466.000000
Electronics       35051
Clothing          41351
Home & Kitchen    38847
Sports            32975
dtype: int64
Jan    11880
Feb    12750
Mar    12440
Apr    14267
May     7587
Jun    17719
Jul    13386
Aug     5992
Sep    11847
Oct    12033
Nov    14952
Dec    13371
dtype: int64
     Electronics  Clothing  Home & Kitchen  Sports  Total Sales  Growth Rate
Jan         3408      3311            1987    3174        11880          NaN
Feb         3152      3458            3713    2427        12750     7.323232
Mar         4286      4058            2713    1383        12440    -2.431373
Apr         3325      4882            4910    1150        14267    14.686495
May         2369      1158            2155    1905         7587   -46.821336
Jun         4094      4512            4673    4440        17719   133.544220
Jul         1499      4775            3070    4042        13386   -24.453976
Aug         1785      1791            1090    1326         5992   -55.236815
Sep         3850      2594            3069    2334        11847    97.713618
Oct         2662      2636            3986    2749        12033     1.570018
Nov         2188      4649            3649    4466        14952    24.258290
Dec         2433      3527            3832    3579        13371   -10.573836
```

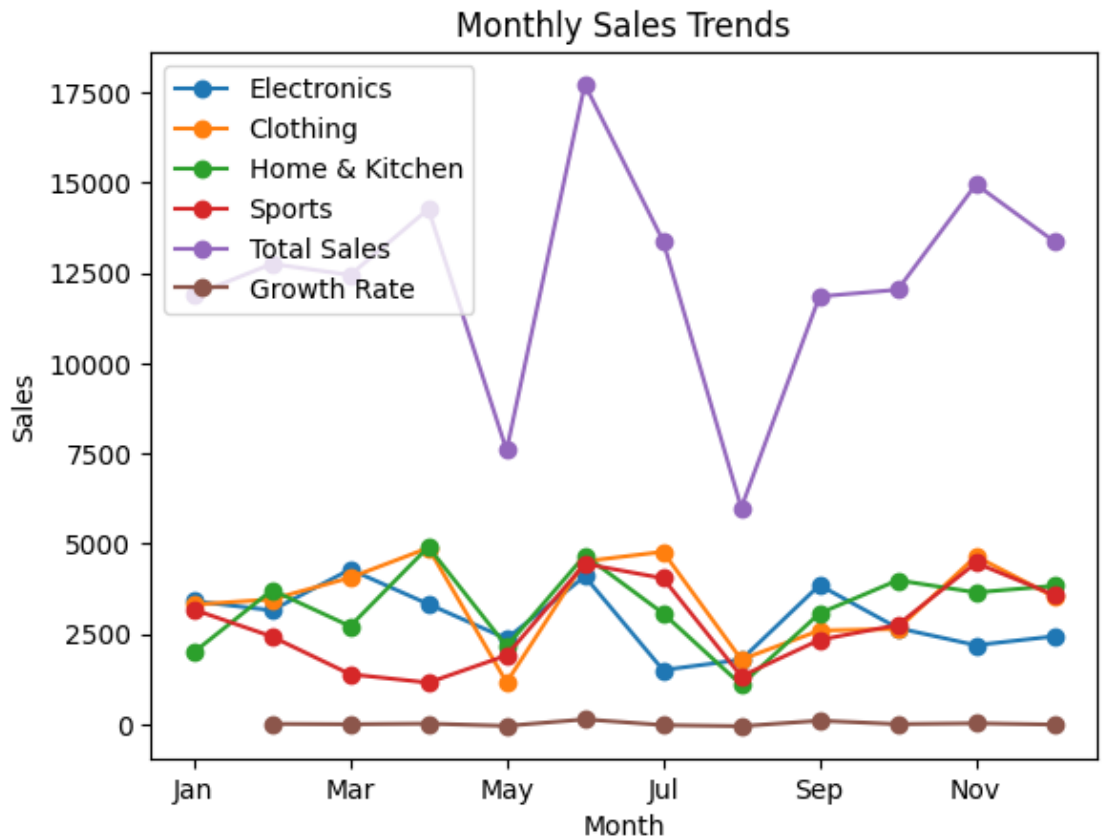
[8]:

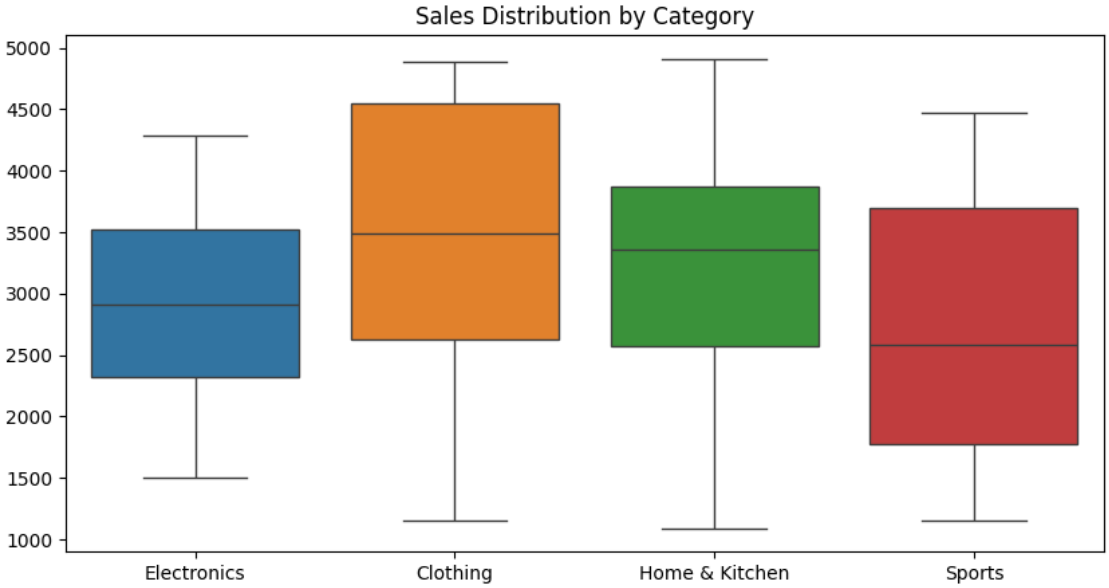
	Electronics	Clothing	Home & Kitchen	Sports	Total Sales	Growth Rate
Jan	3067.2	3311	1987	3174	11880	NaN
Feb	2836.8	3458	3713	2427	12750	7.323232
Mar	3857.4	4058	2713	1383	12440	-2.431373
Apr	2992.5	4882	4910	1150	14267	14.686495
May	2132.1	1158	2155	1905	7587	-46.821336
Jun	3684.6	4512	4673	4440	17719	133.544220
Jul	1349.1	4775	3070	4042	13386	-24.453976
Aug	1606.5	1791	1090	1326	5992	-55.236815
Sep	3465.0	2594	3069	2334	11847	97.713618
Oct	2395.8	2636	3986	2749	12033	1.570018
Nov	1969.2	4649	3649	4466	14952	24.258290
Dec	2189.7	3527	3832	3579	13371	-10.573836

[9]:

```python
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 6))
df.plot(kind='line', marker='o', title='Monthly Sales Trends')
plt.xlabel("Month")
plt.ylabel("Sales")
plt.legend(loc='upper left')
plt.show()

# Box plots
plt.figure(figsize=(10, 5))
sns.boxplot(data=df[categories])
plt.title("Sales Distribution by Category")
plt.show()
```

```
<Figure size 1200x600 with 0 Axes>
```


Monthly Sales Trends
Electronics
Clothing
Home & Kitchen
Sports
Total Sales
Growth Rate
Sales
17500
15000
12500
10000
7500
5000
2500
0
Jan
Mar
May
Jul
Sep
Nov
Month


Sales Distribution by Category
5000
4500
4000
3500
3000
2500
2000
1500
1000
Electronics
Clothing
Home & Kitchen
Sports

[10]:
```
array = np.array([[1, -2, 3], [-4, 5, -6]])
print(np.abs(array))
print(np.percentile(array, [25, 50, 75], axis=None))
print(np.percentile(array, [25, 50, 75], axis=0))
print(np.percentile(array, [25, 50, 75], axis=1))
print(np.mean(array))
print(np.median(array))
print(np.std(array))
```

```
[[1 2 3]
 [4 5 6]]
[-3.5 -0.5  2.5]
[[-2.75 -0.25 -3.75]
 [-1.5   1.5  -1.5 ]
 [-0.25  3.25  0.75]]
[[-0.5 -5. ]
 [ 1.  -4. ]
 [ 2.   0.5]]
-0.5
-0.5
3.8622100754188224
```

[11]:
```
a = np.array([-1.8, -1.6, -0.5, 0.5, 1.6, 1.8, 3.0])
print(np.floor(a))
print(np.ceil(a))
print(np.trunc(a))
print(np.round(a))
```

```
[-2. -2. -1.  0.  1.  1.  3.]
[-1. -1. -0.  1.  2.  2.  3.]
[-1. -1. -0.  0.  1.  1.  3.]
[-2. -2. -0.  0.  2.  2.  3.]
```

[12]:
```
lst = [1, 2, 3, 4, 5]
temp = lst[1]
lst[1] = lst[3]
lst[3] = temp
print("Swapped List:", lst)
```

```
Swapped List: [1, 4, 3, 2, 5]
```

[13]:
```
s = {10, 20, 30, 40, 50}
sl = list(s)
sl[0], sl[4] = sl[4], sl[0]
ss = set(sl)
print("Swapped Set:", ss)
```

```
Swapped Set: {40, 10, 50, 20, 30}
```

assign8

May 18, 2025

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix,␣
↪accuracy_score
from sklearn.preprocessing import StandardScaler

iris = datasets.load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
↪random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

multi_model = LogisticRegression(multi_class='ovr', solver='liblinear')
multi_model.fit(X_train, y_train)

y_pred = multi_model.predict(X_test)

print("\nMulticlass Classification Results:")
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
print("\nAccuracy:", accuracy_score(y_test, y_pred))
```

```
Multiclass Classification Results:
Confusion Matrix:
[[10  0  0]
 [ 0  8  1]
```

```
 [ 0  0 11]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      0.89      0.94         9
           2       0.92      1.00      0.96        11

    accuracy                           0.97        30
   macro avg       0.97      0.96      0.97        30
weighted avg       0.97      0.97      0.97        30


Accuracy: 0.9666666666666667
```

```
c:\Users\Jahid\AppData\Local\Programs\Python\Python313\Lib\site-
packages\sklearn\linear_model\_logistic.py:1256: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.7. Use
OneVsRestClassifier(LogisticRegression(..)) instead. Leave it to its default
value to avoid this warning.
  warnings.warn(
```

Assignment9

May 18, 2025

Q1. Write a unique paragraph (5-6 sentences) about your favorite topic (e.g., sports, technology, food, books, etc.). 1. Convert text to lowercase and remove punctua on. 2. Tokenize the text into words and sentences. 3. Remove stopwords (using NLTK's stopwords list). 4. Display word frequency distribu on (excluding stopwords).

[1]:
```
!pip install nltk==3.8.1
import nltk
nltk.download('punkt')
print(nltk.__version__)
```

```
Requirement already satisfied: nltk==3.8.1 in
c:\users\jahid\appdata\local\programs\python\python313\lib\site-packages (3.8.1)
Requirement already satisfied: click in
c:\users\jahid\appdata\roaming\python\python313\site-packages (from nltk==3.8.1)
(8.1.8)
Requirement already satisfied: joblib in
c:\users\jahid\appdata\local\programs\python\python313\lib\site-packages (from
nltk==3.8.1) (1.4.2)
Requirement already satisfied: regex>=2021.8.3 in
c:\users\jahid\appdata\roaming\python\python313\site-packages (from nltk==3.8.1)
(2024.11.6)
Requirement already satisfied: tqdm in
c:\users\jahid\appdata\roaming\python\python313\site-packages (from nltk==3.8.1)
(4.67.1)
Requirement already satisfied: colorama in
c:\users\jahid\appdata\roaming\python\python313\site-packages (from
click->nltk==3.8.1) (0.4.6)


[notice] A new release of pip is available: 25.0.1 -> 25.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
3.8.1
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\Jahid\AppData\Roaming\nltk_data…
[nltk_data]   Package punkt is already up-to-date!
```

[2]:
```
import string
```

```
text = """Technology is fascinating because it constantly evolves and changes␣
↪the way we live. From smartphones to artificial intelligence, every␣
↪invention makes life more efficient and connected. I love exploring the␣
↪latest gadgets and learning how they work. Reading about new discoveries in␣
↪robotics and space travel excites me. It feels like technology opens doors␣
↪to endless possibilities and future adventures."""

text_lower = text.lower()

text_no_punct = text_lower.translate(str.maketrans('', '', string.punctuation))

print(text_no_punct)
```

```
technology is fascinating because it constantly evolves and changes the way we
live from smartphones to artificial intelligence every invention makes life more
efficient and connected i love exploring the latest gadgets and learning how
they work reading about new discoveries in robotics and space travel excites me
it feels like technology opens doors to endless possibilities and future
adventures
```

[3]:
```
import nltk
nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\Jahid\AppData\Roaming\nltk_data…
[nltk_data]   Package punkt is already up-to-date!
```

[3]: True

[4]:
```
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\Jahid\AppData\Roaming\nltk_data…
[nltk_data]   Package punkt_tab is already up-to-date!
```

[4]: True

[5]:
```
from nltk.tokenize import word_tokenize, sent_tokenize

words = word_tokenize(text_no_punct)

sentences = sent_tokenize(text_no_punct)

print(words)
print(sentences)
```

```
['technology', 'is', 'fascinating', 'because', 'it', 'constantly', 'evolves',
'and', 'changes', 'the', 'way', 'we', 'live', 'from', 'smartphones', 'to',
'artificial', 'intelligence', 'every', 'invention', 'makes', 'life', 'more',
```

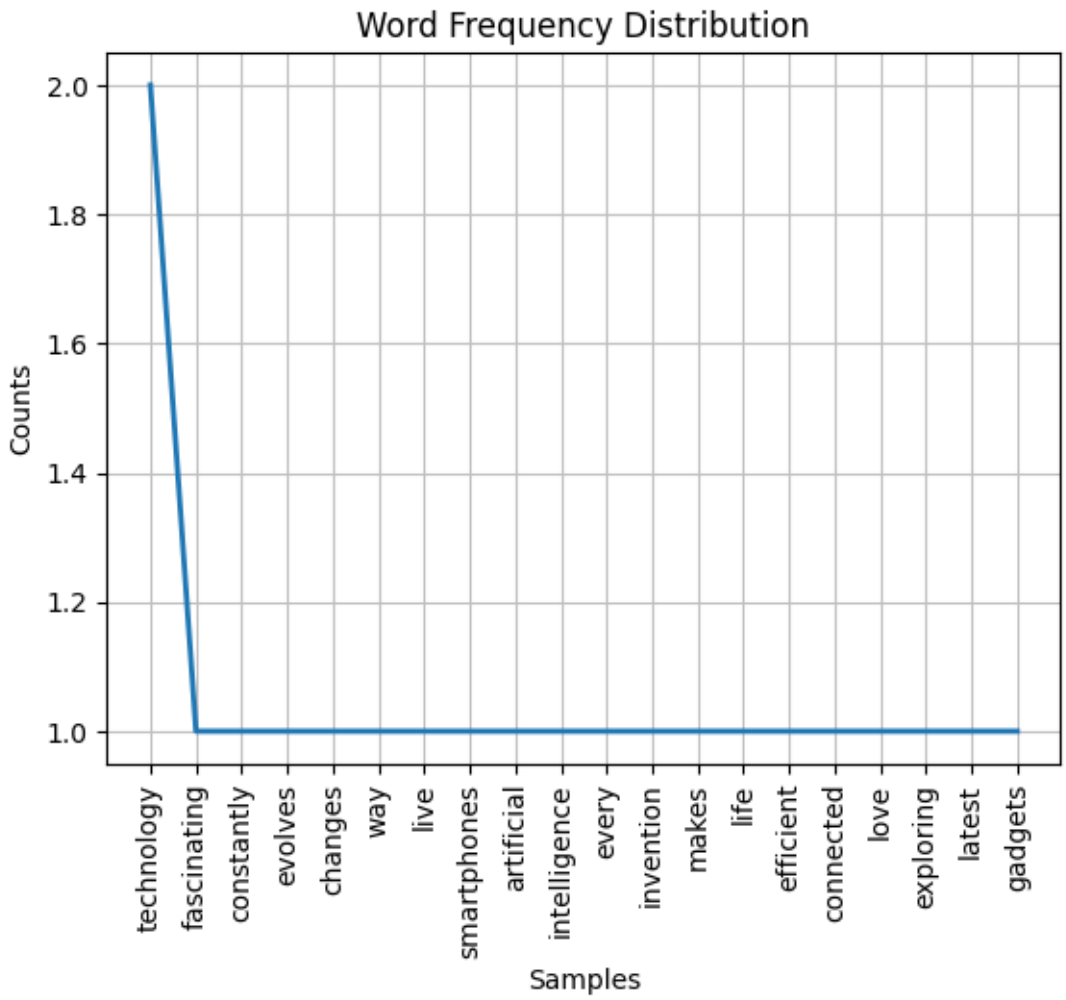
```
'efficient', 'and', 'connected', 'i', 'love', 'exploring', 'the', 'latest',
'gadgets', 'and', 'learning', 'how', 'they', 'work', 'reading', 'about', 'new',
'discoveries', 'in', 'robotics', 'and', 'space', 'travel', 'excites', 'me',
'it', 'feels', 'like', 'technology', 'opens', 'doors', 'to', 'endless',
'possibilities', 'and', 'future', 'adventures']
['technology is fascinating because it constantly evolves and changes the way we
live from smartphones to artificial intelligence every invention makes life more
efficient and connected i love exploring the latest gadgets and learning how
they work reading about new discoveries in robotics and space travel excites me
it feels like technology opens doors to endless possibilities and future
adventures']
```

[6]:
```python
nltk.download('stopwords')
from nltk.corpus import stopwords

stop_words = set(stopwords.words('english'))
filtered_words = [word for word in words if word not in stop_words]

print(filtered_words)
```

```
['technology', 'fascinating', 'constantly', 'evolves', 'changes', 'way', 'live',
'smartphones', 'artificial', 'intelligence', 'every', 'invention', 'makes',
'life', 'efficient', 'connected', 'love', 'exploring', 'latest', 'gadgets',
'learning', 'work', 'reading', 'new', 'discoveries', 'robotics', 'space',
'travel', 'excites', 'feels', 'like', 'technology', 'opens', 'doors', 'endless',
'possibilities', 'future', 'adventures']
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\Jahid\AppData\Roaming\nltk_data…
[nltk_data]   Package stopwords is already up-to-date!
```

[7]:
```python
from nltk.probability import FreqDist

fdist = FreqDist(filtered_words)

fdist.plot(20, title="Word Frequency Distribution")
```



[7]:
```
<Axes: title={'center': 'Word Frequency Distribution'}, xlabel='Samples',
ylabel='Counts'>
```

Q2: Stemming and Lemmatization 1. Take the tokenized words from Question 1 (after stopword removal). 2. Apply stemming using NLTK's PorterStemmer and LancasterStemmer. 3. Apply lemmatization using NLTK's WordNetLemmatizer. 4. Compare and display results of both techniques.

[8]:
```
filtered_words
```

[8]:
```
['technology',
 'fascinating',
 'constantly',
 'evolves',
 'changes',
 'way',
```

```
 'live',
 'smartphones',
 'artificial',
 'intelligence',
 'every',
 'invention',
 'makes',
 'life',
 'efficient',
 'connected',
 'love',
 'exploring',
 'latest',
 'gadgets',
 'learning',
 'work',
 'reading',
 'new',
 'discoveries',
 'robotics',
 'space',
 'travel',
 'excites',
 'feels',
 'like',
 'technology',
 'opens',
 'doors',
 'endless',
 'possibilities',
 'future',
 'adventures']
```

[9]:
```
from nltk.stem import PorterStemmer,LancasterStemmer

ps=PorterStemmer()
ls=LancasterStemmer()

porter_stemmed=[ps.stem(word) for word in filtered_words]
lancaster_stemmed=[ls.stem(word) for word in filtered_words]

print(porter_stemmed)
print(lancaster_stemmed)
```

```
['technolog', 'fascin', 'constantli', 'evolv', 'chang', 'way', 'live',
'smartphon', 'artifici', 'intellig', 'everi', 'invent', 'make', 'life',
'effici', 'connect', 'love', 'explor', 'latest', 'gadget', 'learn', 'work',
'read', 'new', 'discoveri', 'robot', 'space', 'travel', 'excit', 'feel', 'like',
```

```
'technolog', 'open', 'door', 'endless', 'possibl', 'futur', 'adventur']
['technolog', 'fascin', 'const', 'evolv', 'chang', 'way', 'liv', 'smartphon',
'art', 'intellig', 'every', 'inv', 'mak', 'lif', 'efficy', 'connect', 'lov',
'expl', 'latest', 'gadget', 'learn', 'work', 'read', 'new', 'discovery',
'robot', 'spac', 'travel', 'excit', 'feel', 'lik', 'technolog', 'op', 'door',
'endless', 'poss', 'fut', 'adv']
```

[10]:
```
from nltk.stem import WordNetLemmatizer
nltk.download('wordnet')

lemmatizer=WordNetLemmatizer()

lemmatized_words=[lemmatizer.lemmatize(word) for word in filtered_words]

print(lemmatized_words)
```

```
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\Jahid\AppData\Roaming\nltk_data…
[nltk_data]   Package wordnet is already up-to-date!
```

```
['technology', 'fascinating', 'constantly', 'evolves', 'change', 'way', 'live',
'smartphones', 'artificial', 'intelligence', 'every', 'invention', 'make',
'life', 'efficient', 'connected', 'love', 'exploring', 'latest', 'gadget',
'learning', 'work', 'reading', 'new', 'discovery', 'robotics', 'space',
'travel', 'excites', 'feel', 'like', 'technology', 'open', 'door', 'endless',
'possibility', 'future', 'adventure']
```

Q3. Regular Expressions and Text Spliting 1. Take their original text from Question 1. 2. Use regular expressions to: a. Extract all words with more than 5 le ers. b. Extract all numbers (if any exist in their text). c. Extract all capitalized words. 3. Use text spliting techniques to: a. Split the text into words containing only alphabets (removing digits and special characters). b. Extract words starting with a vowel.

[11]:
```
text = "Technology is fascinating because it constantly evolves and changes the␣
↪way we live. Smartphones, Artificial Intelligence, and every new invention␣
↪make life more efficient and connected. I love exploring the latest gadgets␣
↪and learning how they work. Reading about new discoveries in robotics and␣
↪space travel excites me. It feels like Technology opens doors to endless␣
↪possibilities and future adventures."
```

[12]:
```
import re
```

[13]:
```
words_more_than_5 = re.findall(r'\b\w{6,}\b', text)
print("Words with more than 5 letters:", words_more_than_5)
```

```
Words with more than 5 letters: ['Technology', 'fascinating', 'because',
'constantly', 'evolves', 'changes', 'Smartphones', 'Artificial', 'Intelligence',
'invention', 'efficient', 'connected', 'exploring', 'latest', 'gadgets',
'learning', 'Reading', 'discoveries', 'robotics', 'travel', 'excites',
'Technology', 'endless', 'possibilities', 'future', 'adventures']
```

[14]:
```
numbers = re.findall(r'\b\d+\b', text)
print("Numbers:", numbers)
```

```
Numbers: []
```

[15]:
```
capitalized_words = re.findall(r'\b[A-Z][a-z]*\b', text)
print("Capitalized Words:", capitalized_words)
```

```
Capitalized Words: ['Technology', 'Smartphones', 'Artificial', 'Intelligence',
'I', 'Reading', 'It', 'Technology']
```

[16]:
```
words_only_alpha = re.findall(r'\b[a-zA-Z]+\b', text)
print("Words with only alphabets:", words_only_alpha)
```

```
Words with only alphabets: ['Technology', 'is', 'fascinating', 'because', 'it',
'constantly', 'evolves', 'and', 'changes', 'the', 'way', 'we', 'live',
'Smartphones', 'Artificial', 'Intelligence', 'and', 'every', 'new', 'invention',
'make', 'life', 'more', 'efficient', 'and', 'connected', 'I', 'love',
'exploring', 'the', 'latest', 'gadgets', 'and', 'learning', 'how', 'they',
'work', 'Reading', 'about', 'new', 'discoveries', 'in', 'robotics', 'and',
'space', 'travel', 'excites', 'me', 'It', 'feels', 'like', 'Technology',
'opens', 'doors', 'to', 'endless', 'possibilities', 'and', 'future',
'adventures']
```

[17]:
```
words_starting_vowel = re.findall(r'\b[AEIOUaeiou][a-zA-Z]*\b', text)
print("Words starting with a vowel:", words_starting_vowel)
```

```
Words starting with a vowel: ['is', 'it', 'evolves', 'and', 'Artificial',
'Intelligence', 'and', 'every', 'invention', 'efficient', 'and', 'I',
'exploring', 'and', 'about', 'in', 'and', 'excites', 'It', 'opens', 'endless',
'and', 'adventures']
```

[18]:
```
print(nltk.__version__)
```

```
3.8.1
```

Q4. Custom Tokenization & Regex-based Text Cleaning 1. Take original text from Question 1. 2. Write a custom tokenization function that: a. Removes punctuation and special symbols, but keeps contractions (e.g., "isn't" should not be split into "is" and "n't"). b. Handles hyphenated words as a single token (e.g., "state-of-the-art" remains a single token). c. Tokenizes numbers separately but keeps decimal numbers intact (e.g., "3.14" should remain as is). 3. Use Regex Substitutions (re.sub) to: a. Replace email addresses with '' placeholder. b. Replace URLs with '' placeholder. c. Replace phone numbers (formats: 123-456-7890 or +91 9876543210) with '' placeholder.

[19]:
```
import re

text = """I love state-of-the-art technology! It's amazing how quickly AI,␣
↪machine-learning, and robotics are evolving.
Visit https://example.com for more details or contact me at john.doe123@example.
↪com or +91 9876543210."""
```

```python
def custom_tokenizer(text):
    text = re.sub(r"[^\w\s\.\-']", '', text)

    tokens = text.split()

    return tokens

def clean_text(text):
    text = re.sub(r'\b[\w\.-]+@[\w\.-]+\.\w{2,4}\b', '<EMAIL>', text)

    text = re.sub(r'https?://\S+|www\.\S+', '<URL>', text)

    text = re.sub(r'(\+?\d{1,3}[-\s]?\d{10}|\d{3}-\d{3}-\d{4})', '<PHONE>',␣
 ↪text)

    return text

cleaned_text = clean_text(text)
print("Cleaned Text:\n", cleaned_text)

tokens = custom_tokenizer(cleaned_text)
print("\nCustom Tokens:\n", tokens)
```

```
Cleaned Text:
 I love state-of-the-art technology! It's amazing how quickly AI, machine-
learning, and robotics are evolving.
Visit <URL> for more details or contact me at <EMAIL> or <PHONE>.

Custom Tokens:
 ['I', 'love', 'state-of-the-art', 'technology', "It's", 'amazing', 'how',
'quickly', 'AI', 'machine-learning', 'and', 'robotics', 'are', 'evolving.',
'Visit', 'URL', 'for', 'more', 'details', 'or', 'contact', 'me', 'at', 'EMAIL',
'or', 'PHONE.']
```

Assignment10

May 18, 2025

Q1. Write a unique paragraph (5-6 sentences) about your favorite topic (e.g., sports, technology, food, books, etc.). 1. Convert text to lowercase and remove punctua on using re. 2. Tokenize the text into words and sentences. 3. Split using split() and word_tokenize() and compare how Python split and NLTK's word_tokenize() differ. 4. Remove stopwords (using NLTK's stopwords list). 5. Display word frequency distribu on (excluding stopwords).

[22]:
```
import re

text = """ChatGPT is an advanced language model developed by OpenAI that uses␣
↪artificial intelligence to understand and generate human-like text. It is␣
↪based on the GPT (Generative Pre-trained Transformer) architecture and is␣
↪capable of answering questions, writing code, generating creative content,␣
↪translating languages, summarizing text, and more. ChatGPT has been trained␣
↪on a large amount of text data from the internet, allowing it to respond to␣
↪a wide range of topics with contextual understanding. It is widely used in␣
↪education, software development, customer service, and personal productivity.
↪ With continuous improvements, ChatGPT is becoming a powerful tool for both␣
↪casual users and professionals."""


text_lower = text.lower()


text_clean = re.sub(r'[^\w\s]', '', text_lower)

print(text_clean)
```

```
chatgpt is an advanced language model developed by openai that uses artificial
intelligence to understand and generate humanlike text it is based on the gpt
generative pretrained transformer architecture and is capable of answering
questions writing code generating creative content translating languages
summarizing text and more chatgpt has been trained on a large amount of text
data from the internet allowing it to respond to a wide range of topics with
contextual understanding it is widely used in education software development
customer service and personal productivity with continuous improvements chatgpt
is becoming a powerful tool for both casual users and professionals
```

[23]:
```python
import nltk
nltk.download('punkt_tab')
from nltk.tokenize import word_tokenize, sent_tokenize

sentences = sent_tokenize(text)
print("Sentences:", sentences)

words = word_tokenize(text_clean)
print("Words:", words)
```

```
Sentences: ['ChatGPT is an advanced language model developed by OpenAI that uses
artificial intelligence to understand and generate human-like text.', 'It is
based on the GPT (Generative Pre-trained Transformer) architecture and is
capable of answering questions, writing code, generating creative content,
translating languages, summarizing text, and more.', 'ChatGPT has been trained
on a large amount of text data from the internet, allowing it to respond to a
wide range of topics with contextual understanding.', 'It is widely used in
education, software development, customer service, and personal productivity.',
'With continuous improvements, ChatGPT is becoming a powerful tool for both
casual users and professionals.']
Words: ['chatgpt', 'is', 'an', 'advanced', 'language', 'model', 'developed',
'by', 'openai', 'that', 'uses', 'artificial', 'intelligence', 'to',
'understand', 'and', 'generate', 'humanlike', 'text', 'it', 'is', 'based', 'on',
'the', 'gpt', 'generative', 'pretrained', 'transformer', 'architecture', 'and',
'is', 'capable', 'of', 'answering', 'questions', 'writing', 'code',
'generating', 'creative', 'content', 'translating', 'languages', 'summarizing',
'text', 'and', 'more', 'chatgpt', 'has', 'been', 'trained', 'on', 'a', 'large',
'amount', 'of', 'text', 'data', 'from', 'the', 'internet', 'allowing', 'it',
'to', 'respond', 'to', 'a', 'wide', 'range', 'of', 'topics', 'with',
'contextual', 'understanding', 'it', 'is', 'widely', 'used', 'in', 'education',
'software', 'development', 'customer', 'service', 'and', 'personal',
'productivity', 'with', 'continuous', 'improvements', 'chatgpt', 'is',
'becoming', 'a', 'powerful', 'tool', 'for', 'both', 'casual', 'users', 'and',
'professionals']
```

```
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\Jahid\AppData\Roaming\nltk_data…
[nltk_data]   Package punkt_tab is already up-to-date!
```

[24]:
```python
split_words = text_clean.split()
print("Split words:", split_words)


tokenized_words = word_tokenize(text_clean)
print("Tokenized words:", tokenized_words)
```

```
Split words: ['chatgpt', 'is', 'an', 'advanced', 'language', 'model',
'developed', 'by', 'openai', 'that', 'uses', 'artificial', 'intelligence', 'to',
'understand', 'and', 'generate', 'humanlike', 'text', 'it', 'is', 'based', 'on',
```

```
'the', 'gpt', 'generative', 'pretrained', 'transformer', 'architecture', 'and',
'is', 'capable', 'of', 'answering', 'questions', 'writing', 'code',
'generating', 'creative', 'content', 'translating', 'languages', 'summarizing',
'text', 'and', 'more', 'chatgpt', 'has', 'been', 'trained', 'on', 'a', 'large',
'amount', 'of', 'text', 'data', 'from', 'the', 'internet', 'allowing', 'it',
'to', 'respond', 'to', 'a', 'wide', 'range', 'of', 'topics', 'with',
'contextual', 'understanding', 'it', 'is', 'widely', 'used', 'in', 'education',
'software', 'development', 'customer', 'service', 'and', 'personal',
'productivity', 'with', 'continuous', 'improvements', 'chatgpt', 'is',
'becoming', 'a', 'powerful', 'tool', 'for', 'both', 'casual', 'users', 'and',
'professionals']
Tokenized words: ['chatgpt', 'is', 'an', 'advanced', 'language', 'model',
'developed', 'by', 'openai', 'that', 'uses', 'artificial', 'intelligence', 'to',
'understand', 'and', 'generate', 'humanlike', 'text', 'it', 'is', 'based', 'on',
'the', 'gpt', 'generative', 'pretrained', 'transformer', 'architecture', 'and',
'is', 'capable', 'of', 'answering', 'questions', 'writing', 'code',
'generating', 'creative', 'content', 'translating', 'languages', 'summarizing',
'text', 'and', 'more', 'chatgpt', 'has', 'been', 'trained', 'on', 'a', 'large',
'amount', 'of', 'text', 'data', 'from', 'the', 'internet', 'allowing', 'it',
'to', 'respond', 'to', 'a', 'wide', 'range', 'of', 'topics', 'with',
'contextual', 'understanding', 'it', 'is', 'widely', 'used', 'in', 'education',
'software', 'development', 'customer', 'service', 'and', 'personal',
'productivity', 'with', 'continuous', 'improvements', 'chatgpt', 'is',
'becoming', 'a', 'powerful', 'tool', 'for', 'both', 'casual', 'users', 'and',
'professionals']
```

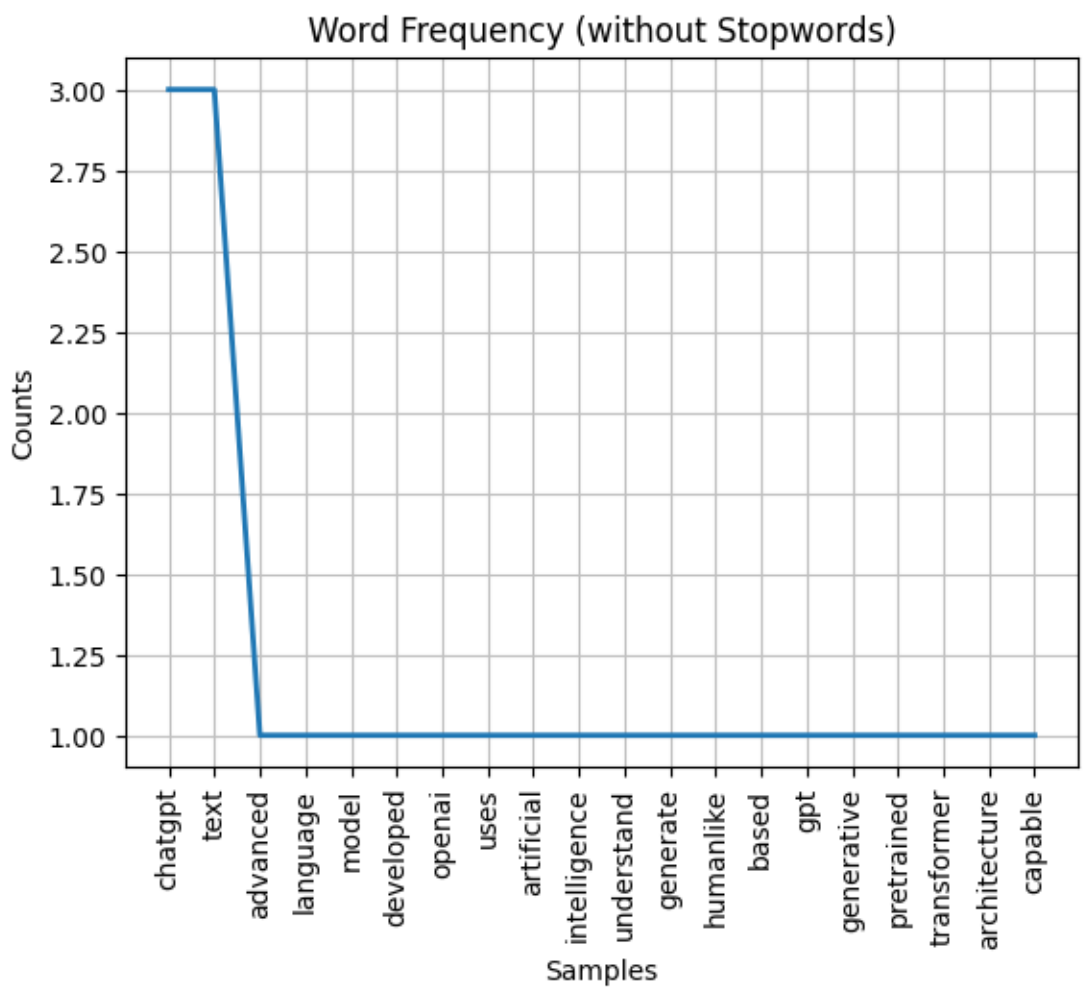
[25]:
```python
from nltk.corpus import stopwords

nltk.download('stopwords')
stop_words = set(stopwords.words('english'))

filtered_words = [word for word in tokenized_words if word not in stop_words]
print("Filtered Words:", filtered_words)
```

```
Filtered Words: ['chatgpt', 'advanced', 'language', 'model', 'developed',
'openai', 'uses', 'artificial', 'intelligence', 'understand', 'generate',
'humanlike', 'text', 'based', 'gpt', 'generative', 'pretrained', 'transformer',
'architecture', 'capable', 'answering', 'questions', 'writing', 'code',
'generating', 'creative', 'content', 'translating', 'languages', 'summarizing',
'text', 'chatgpt', 'trained', 'large', 'amount', 'text', 'data', 'internet',
'allowing', 'respond', 'wide', 'range', 'topics', 'contextual', 'understanding',
'widely', 'used', 'education', 'software', 'development', 'customer', 'service',
'personal', 'productivity', 'continuous', 'improvements', 'chatgpt', 'becoming',
'powerful', 'tool', 'casual', 'users', 'professionals']
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\Jahid\AppData\Roaming\nltk_data…
[nltk_data]   Package stopwords is already up-to-date!
```

[26]:
```
from nltk import FreqDist

fdist = FreqDist(filtered_words)

fdist.plot(20, title="Word Frequency (without Stopwords)")
```

[26]:
```
<Axes: title={'center': 'Word Frequency (without Stopwords)'}, xlabel='Samples',
ylabel='Counts'>
```



Q2. Using the same paragraph from Q1: 1. Extract all words with only alphabets using re.findall() 2. Remove stop words using NLTK's stopword list 3. Perform stemming with PorterStemmer 4. Perform lemma za on with WordNetLemma zer 5. Compare the stemmed and lemma zed outputs and explain when you'd prefer one over the other.

[27]:
```
import re
```

```
text_lower = text.lower()

alphabetic_words = re.findall(r'\b[a-zA-Z]+\b', text_lower)
print("Alphabetic Words:", alphabetic_words)
```

```
Alphabetic Words: ['chatgpt', 'is', 'an', 'advanced', 'language', 'model',
'developed', 'by', 'openai', 'that', 'uses', 'artificial', 'intelligence', 'to',
'understand', 'and', 'generate', 'human', 'like', 'text', 'it', 'is', 'based',
'on', 'the', 'gpt', 'generative', 'pre', 'trained', 'transformer',
'architecture', 'and', 'is', 'capable', 'of', 'answering', 'questions',
'writing', 'code', 'generating', 'creative', 'content', 'translating',
'languages', 'summarizing', 'text', 'and', 'more', 'chatgpt', 'has', 'been',
'trained', 'on', 'a', 'large', 'amount', 'of', 'text', 'data', 'from', 'the',
'internet', 'allowing', 'it', 'to', 'respond', 'to', 'a', 'wide', 'range', 'of',
'topics', 'with', 'contextual', 'understanding', 'it', 'is', 'widely', 'used',
'in', 'education', 'software', 'development', 'customer', 'service', 'and',
'personal', 'productivity', 'with', 'continuous', 'improvements', 'chatgpt',
'is', 'becoming', 'a', 'powerful', 'tool', 'for', 'both', 'casual', 'users',
'and', 'professionals']
```

[28]:
```
from nltk.corpus import stopwords
nltk.download('stopwords')

stop_words = set(stopwords.words('english'))

filtered_words = [word for word in alphabetic_words if word not in stop_words]
print("Filtered Words:", filtered_words)
```

```
Filtered Words: ['chatgpt', 'advanced', 'language', 'model', 'developed',
'openai', 'uses', 'artificial', 'intelligence', 'understand', 'generate',
'human', 'like', 'text', 'based', 'gpt', 'generative', 'pre', 'trained',
'transformer', 'architecture', 'capable', 'answering', 'questions', 'writing',
'code', 'generating', 'creative', 'content', 'translating', 'languages',
'summarizing', 'text', 'chatgpt', 'trained', 'large', 'amount', 'text', 'data',
'internet', 'allowing', 'respond', 'wide', 'range', 'topics', 'contextual',
'understanding', 'widely', 'used', 'education', 'software', 'development',
'customer', 'service', 'personal', 'productivity', 'continuous', 'improvements',
'chatgpt', 'becoming', 'powerful', 'tool', 'casual', 'users', 'professionals']
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\Jahid\AppData\Roaming\nltk_data…
[nltk_data]   Package stopwords is already up-to-date!
```

[29]:
```
from nltk.stem import PorterStemmer

porter = PorterStemmer()

stemmed_words = [porter.stem(word) for word in filtered_words]
print("Stemmed Words:", stemmed_words)
```

```
Stemmed Words: ['chatgpt', 'advanc', 'languag', 'model', 'develop', 'openai',
'use', 'artifici', 'intellig', 'understand', 'gener', 'human', 'like', 'text',
'base', 'gpt', 'gener', 'pre', 'train', 'transform', 'architectur', 'capabl',
'answer', 'question', 'write', 'code', 'gener', 'creativ', 'content',
'translat', 'languag', 'summar', 'text', 'chatgpt', 'train', 'larg', 'amount',
'text', 'data', 'internet', 'allow', 'respond', 'wide', 'rang', 'topic',
'contextu', 'understand', 'wide', 'use', 'educ', 'softwar', 'develop', 'custom',
'servic', 'person', 'product', 'continu', 'improv', 'chatgpt', 'becom', 'power',
'tool', 'casual', 'user', 'profession']
```

[30]:
```
from nltk.stem import WordNetLemmatizer
nltk.download('wordnet')
nltk.download('omw-1.4')

lemmatizer = WordNetLemmatizer()

lemmatized_words = [lemmatizer.lemmatize(word) for word in filtered_words]
print("Lemmatized Words:", lemmatized_words)
```

```
Lemmatized Words: ['chatgpt', 'advanced', 'language', 'model', 'developed',
'openai', 'us', 'artificial', 'intelligence', 'understand', 'generate', 'human',
'like', 'text', 'based', 'gpt', 'generative', 'pre', 'trained', 'transformer',
'architecture', 'capable', 'answering', 'question', 'writing', 'code',
'generating', 'creative', 'content', 'translating', 'language', 'summarizing',
'text', 'chatgpt', 'trained', 'large', 'amount', 'text', 'data', 'internet',
'allowing', 'respond', 'wide', 'range', 'topic', 'contextual', 'understanding',
'widely', 'used', 'education', 'software', 'development', 'customer', 'service',
'personal', 'productivity', 'continuous', 'improvement', 'chatgpt', 'becoming',
'powerful', 'tool', 'casual', 'user', 'professional']

[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\Jahid\AppData\Roaming\nltk_data…
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to
[nltk_data]     C:\Users\Jahid\AppData\Roaming\nltk_data…
[nltk_data]   Package omw-1.4 is already up-to-date!
```

Q3. Choose 3 short texts of your own (e.g., different news headlines, product reviews). 1. Use CountVectorizer to generate the Bag of Words representa on. 2. Use TfidfVectorizer to compute TF-IDF scores. 3. Print and interpret the top 3 keywords from each text using TF-IDF.

[31]:
```
texts = [
    "SpaceX Launches Falcon 9 Rocket With 28 Starlink Satellites Into Orbit␣
 ↪From Florida",
    "US Galaxy S22, S22+ and S22 Ultra One UI 7 Android 15 update out now",
    "Rockstar Games sets new GTA 6 launch date, focuses on quality improvements"
]
```

[32]:
```python
from sklearn.feature_extraction.text import CountVectorizer

count_vectorizer = CountVectorizer()

count_matrix = count_vectorizer.fit_transform(texts)


print("Vocabulary:", count_vectorizer.get_feature_names_out())

print("Bag of Words Matrix:\n", count_matrix.toarray())
```

```
Vocabulary: ['15' '28' 'and' 'android' 'date' 'falcon' 'florida' 'focuses'
'from'
 'galaxy' 'games' 'gta' 'improvements' 'into' 'launch' 'launches' 'new'
 'now' 'on' 'one' 'orbit' 'out' 'quality' 'rocket' 'rockstar' 's22'
 'satellites' 'sets' 'spacex' 'starlink' 'ui' 'ultra' 'update' 'us' 'with']
Bag of Words Matrix:
 [[0 1 0 0 0 1 1 0 1 0 0 0 0 1 0 1 0 0 0 0 1 0 0 1 0 0 1 0 1 1 0 0 0 0 1]
 [1 0 1 1 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 1 0 1 0 0 0 3 0 0 0 0 1 1 1 1 0]
 [0 0 0 0 1 0 0 1 0 0 1 1 1 0 1 0 1 0 1 0 0 0 1 0 1 0 0 1 0 0 0 0 0 0 0]]
```

[33]:
```python
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vectorizer = TfidfVectorizer()

tfidf_matrix = tfidf_vectorizer.fit_transform(texts)


feature_names = tfidf_vectorizer.get_feature_names_out()


print("TF-IDF Matrix:\n", tfidf_matrix.toarray())
```

```
TF-IDF Matrix:
 [[0.         0.28867513 0.         0.         0.         0.28867513
  0.28867513 0.         0.28867513 0.         0.         0.
  0.         0.28867513 0.         0.28867513 0.         0.
  0.         0.         0.28867513 0.         0.         0.28867513
  0.         0.         0.28867513 0.         0.28867513 0.28867513
  0.         0.         0.         0.         0.28867513]
 [0.2236068  0.         0.2236068  0.2236068  0.         0.
  0.         0.         0.         0.2236068  0.         0.
  0.         0.         0.         0.         0.         0.2236068
  0.         0.2236068  0.         0.2236068  0.         0.
  0.         0.67082039 0.         0.         0.         0.
  0.2236068  0.2236068  0.2236068  0.2236068  0.        ]
 [0.         0.         0.         0.         0.30151134 0.
  0.         0.30151134 0.         0.         0.30151134 0.30151134
```

```
 0.30151134 0.         0.30151134 0.         0.30151134 0.
 0.30151134 0.         0.         0.         0.30151134 0.
 0.30151134 0.         0.         0.30151134 0.         0.
 0.         0.         0.         0.         0.        ]]
```

[34]:
```python
import numpy as np


for i in range(len(texts)):
    print(f"\nText {i+1}: {texts[i]}")

    row = tfidf_matrix[i].toarray()[0]

    top_indices = row.argsort()[-3:][::-1]

    for idx in top_indices:
        print(f"   {feature_names[idx]} (Score: {row[idx]:.3f})")
```

```
Text 1: SpaceX Launches Falcon 9 Rocket With 28 Starlink Satellites Into Orbit
From Florida
   with (Score: 0.289)
   starlink (Score: 0.289)
   spacex (Score: 0.289)

Text 2: US Galaxy S22, S22+ and S22 Ultra One UI 7 Android 15 update out now
   s22 (Score: 0.671)
   update (Score: 0.224)
   us (Score: 0.224)

Text 3: Rockstar Games sets new GTA 6 launch date, focuses on quality
improvements
   on (Score: 0.302)
   sets (Score: 0.302)
   rockstar (Score: 0.302)
```

Q4. Write 2 short texts (4–6 lines each) describing two different technologies (e.g., AI vs Blockchain). 1. Preprocess and tokenize both texts. 2. Calculate: a. Jaccard Similarity using sets b. Cosine Similarity using TfidfVectorizer + cosine_similarity() c. Analyze which similarity metric gives be er insights in your case

[35]:
```python
import re
import nltk
nltk.download('punkt_tab')
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
nltk.download('punkt')
nltk.download('stopwords')
```

```
text1 = """Artificial Intelligence is a field of computer science that focuses␣
↪on creating machines capable of mimicking human intelligence. It includes␣
↪areas like machine learning, natural language processing, and robotics. AI␣
↪systems are designed to solve problems, make decisions, and learn from␣
↪experience."""

text2 = """Blockchain is a decentralized and secure technology that stores data␣
↪in blocks connected in a chain. It enables transparent, tamper-proof␣
↪transactions without the need for intermediaries. Blockchain powers␣
↪cryptocurrencies like Bitcoin and supports smart contracts."""

def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^\w\s]', '', text)
    tokens = word_tokenize(text)
    tokens = [word for word in tokens if word not in stopwords.words('english')]
    return tokens

tokens1 = preprocess(text1)
tokens2 = preprocess(text2)

print("Tokens 1:", tokens1)
print("Tokens 2:", tokens2)
```

```
Tokens 1: ['artificial', 'intelligence', 'field', 'computer', 'science',
'focuses', 'creating', 'machines', 'capable', 'mimicking', 'human',
'intelligence', 'includes', 'areas', 'like', 'machine', 'learning', 'natural',
'language', 'processing', 'robotics', 'ai', 'systems', 'designed', 'solve',
'problems', 'make', 'decisions', 'learn', 'experience']
Tokens 2: ['blockchain', 'decentralized', 'secure', 'technology', 'stores',
'data', 'blocks', 'connected', 'chain', 'enables', 'transparent', 'tamperproof',
'transactions', 'without', 'need', 'intermediaries', 'blockchain', 'powers',
'cryptocurrencies', 'like', 'bitcoin', 'supports', 'smart', 'contracts']
```

```
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\Jahid\AppData\Roaming\nltk_data…
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\Jahid\AppData\Roaming\nltk_data…
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\Jahid\AppData\Roaming\nltk_data…
[nltk_data]   Package stopwords is already up-to-date!
```

[36]:
```
set1 = set(tokens1)
set2 = set(tokens2)
```

```
jaccard_sim = len(set1 & set2) / len(set1 | set2)
print("Jaccard Similarity:", round(jaccard_sim, 3))
```

```
Jaccard Similarity: 0.02
```

[37]:
```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

corpus = [text1, text2]
vectorizer = TfidfVectorizer(stop_words='english')
tfidf_matrix = vectorizer.fit_transform(corpus)

cos_sim = cosine_similarity(tfidf_matrix[0:1], tfidf_matrix[1:2])
print("Cosine Similarity:", round(cos_sim[0][0], 3))
```

```
Cosine Similarity: 0.018
```

Q5. Write a short review for a product or service. 1. Use TextBlob or VADER to find polarity & subjec vity for each review. 2. Classify reviews into Posi ve / Nega ve / Neutral. 3. Create a word cloud using the wordcloud library for all posi ve reviews

[38]:
```
from textblob import TextBlob

review = """I've been using the Samsung S22 Ultra for a while now, and overall␣
↪it's a fantastic smartphone. The display is stunning, the performance is␣
↪super smooth, and the camera system is one of the best I've ever used. It␣
↪feels premium in the hand and handles everything from gaming to multitasking␣
↪effortlessly. However, the battery life is a bit of a letdown - it doesn't␣
↪always last a full day with heavy use, which is disappointing for a flagship␣
↪at this price point. Also, the One UI 7 update has been taking way too long␣
↪to roll out, which is frustrating for users waiting on new features and␣
↪improvements. Despite these issues, it's still a solid device, especially if␣
↪performance and camera quality are your top priorities."""

blob = TextBlob(review)
polarity = blob.sentiment.polarity
subjectivity = blob.sentiment.subjectivity

print(f"Polarity: {polarity}")
print(f"Subjectivity: {subjectivity}")
```

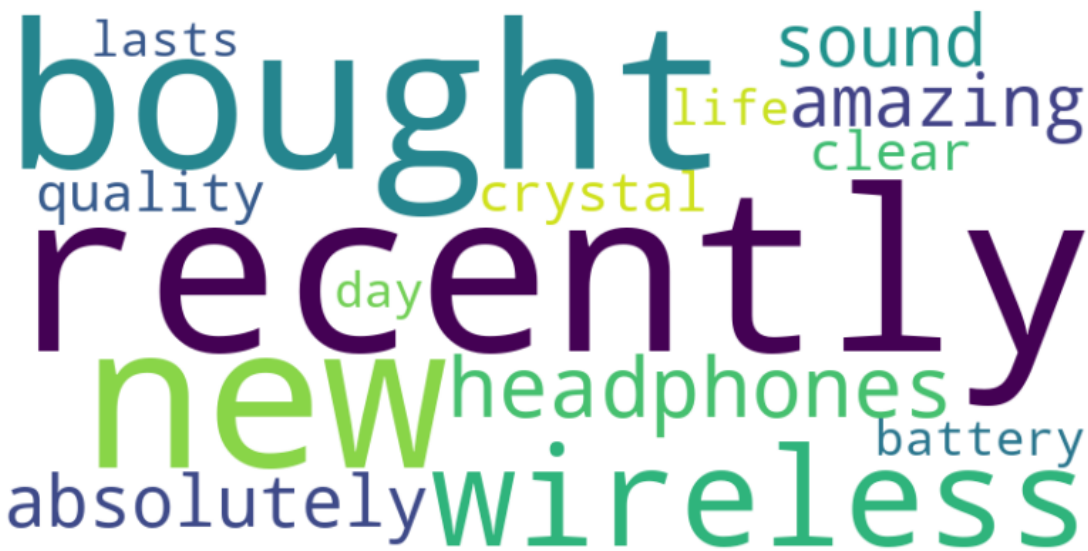
```
Polarity: 0.1481060606060606
Subjectivity: 0.5336174242424243
```

[39]:
```
if polarity > 0:
    sentiment = "Positive"
elif polarity < 0:
    sentiment = "Negative"
else:
```

```
    sentiment = "Neutral"

print("Sentiment:", sentiment)
```

```
Sentiment: Positive
```

[40]:
```
from wordcloud import WordCloud
import matplotlib.pyplot as plt

positive_reviews = """I recently bought the new wireless headphones, and they␣
↪are absolutely amazing!
The sound quality is crystal clear, and the battery life lasts all day."""

wordcloud = WordCloud(width=800, height=400, background_color='white').
↪generate(positive_reviews)

plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.show()
```



Q6. Choose your own paragraph (~100 words) as training data. 1. Tokenize text using Tokenizer() from keras.preprocessing.text 2. Create input sequences and build a simple LSTM or Dense model 3. Train the model and generate 2–3 new lines of text star ng from any seed word you

[ ]:
```

```

```
paragraph = """ChatGPT is an advanced language model developed by OpenAI that␣
↪uses artificial intelligence to understand and generate human-like text. It␣
↪is based on the GPT (Generative Pre-trained Transformer) architecture and is␣
↪capable of answering questions, writing code, generating creative content,␣
↪translating languages, summarizing text, and more. ChatGPT has been trained␣
↪on a large amount of text data from the internet, allowing it to respond to␣
↪a wide range of topics with contextual understanding. It is widely used in␣
↪education, software development, customer service, and personal productivity.
↪ With continuous improvements, ChatGPT is becoming a powerful tool for both␣
↪casual users and professionals."""
```

[ ]:
```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
import numpy as np
```

[ ]:
```
tokenizer = Tokenizer()
tokenizer.fit_on_texts([paragraph])

sequences = []
total_words = len(tokenizer.word_index) + 1

for line in paragraph.split('.'):
    token_list = tokenizer.texts_to_sequences([line])[0]
    for i in range(1, len(token_list)):
        n_gram_sequence = token_list[:i+1]
        sequences.append(n_gram_sequence)


max_sequence_len = max([len(seq) for seq in sequences])
sequences = pad_sequences(sequences, maxlen=max_sequence_len, padding='pre')

X = sequences[:, :-1]
y = sequences[:, -1]
y = np.array(y)
```

[ ]:
```
model = Sequential()
model.add(Embedding(total_words, 10, input_length=max_sequence_len-1))
model.add(LSTM(100))
model.add(Dense(total_words, activation='softmax'))

model.compile(loss='sparse_categorical_crossentropy', optimizer='adam',␣
↪metrics=['accuracy'])
```

[ ]:
```
model.fit(X, y, epochs=200, verbose=1)
```

[ ]:
```
seed_text = "Artificial intelligence"
next_words = 5

for _ in range(next_words):
    token_list = tokenizer.texts_to_sequences([seed_text])[0]
    token_list = pad_sequences([token_list], maxlen=max_sequence_len-1,␣
 ↪padding='pre')
    predicted = model.predict(token_list, verbose=0)
    predicted_word_index = np.argmax(predicted)

    output_word = ''
    for word, index in tokenizer.word_index.items():
        if index == predicted_word_index:
            output_word = word
            break
    seed_text += " " + output_word

print(seed_text)
```